

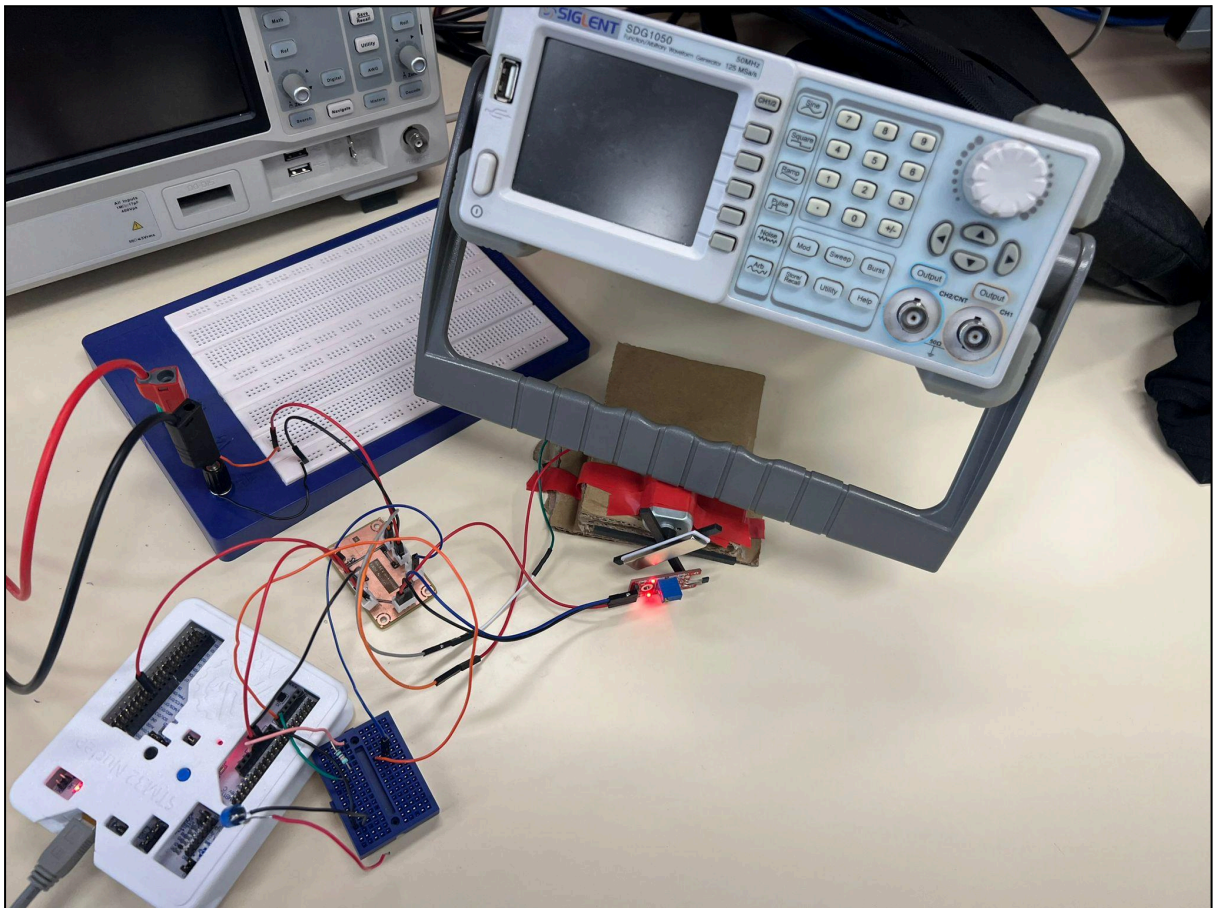
Compte-rendu de Projet S6

Asservissement d'un MCC pour drone

Rappel des objectifs et pertinence du projet

Acquérir de l'expérience sur le déroulement d'un projet impliquant de l'électronique (PCBs, Programmation carte STM32, utilisation imprimante 3D, etc.)

L'objectif du projet est d'asservir un MCC utilisable pour des drones par le biais d'une STM32. Toute valeur ajoutée sera appréciée.



**FADE Lizandre Nicolas / AMAEV Mokhmad-ali / Groupe: JEBRI
ENSEA - 2025**

I - Introduction

I - 1 Contextualisation

Les drones étant petits (ceux commerciaux) les moteurs doivent être de faible dimension et de poids raisonnable. Pour ne pas à avoir à embarquer un onduleur en plus de la batterie (le poids est un facteur important), les moteurs de drones sont quasi-exclusivement des MCC (moteur à courant continu).

Le couple et la vitesse de rotation utilisés pour les moteurs dépendent généralement de la longueur des pâles et du poids du drone. Ces moteurs ont des caractéristiques propres à eux-même.

Les moteurs de voiture, par exemple, ont une vitesse de rotation au niveau des roues de l'ordre de 20 tours.s-1 tandis que celle des drones sont de l'ordre de 50 tours.s-1. En outre, vu que les drones sont légers, les couples mis en jeu sont généralement plus faibles.

L'enjeux aurait été d'asservir un moteur étant plus sensible dynamiquement (couple faible => plus sensible aux couples perturbateurs externes).

I - 2 Cahier des charges

Au départ nous avons décidé du cahier des charges suivant pour un drone fictif :

Vitesse nominale	> 3000tours/min-1
Une précision d'au maximum	5%
Temps de réponse d'au maximum	0.1s
Une tension nominale	5V
Une saturation de courant d'au maximum	2A

Et si possible : [Finalement pas fait]

-Essayer de ne pas trop gaspiller d'énergie

-Commander par STM32 et puis entièrement électriquement [non pertinent d'un point de vue client]

I - 3 Organisation

1) Affectation des tâches

Lizandre s'est concentré principalement sur le codage et l'ajout de fonctionnalités (le potentiomètre, les cartons tenant le moteur, le design des pâles).

Mokmad s'est occupé principalement du design du PCB.

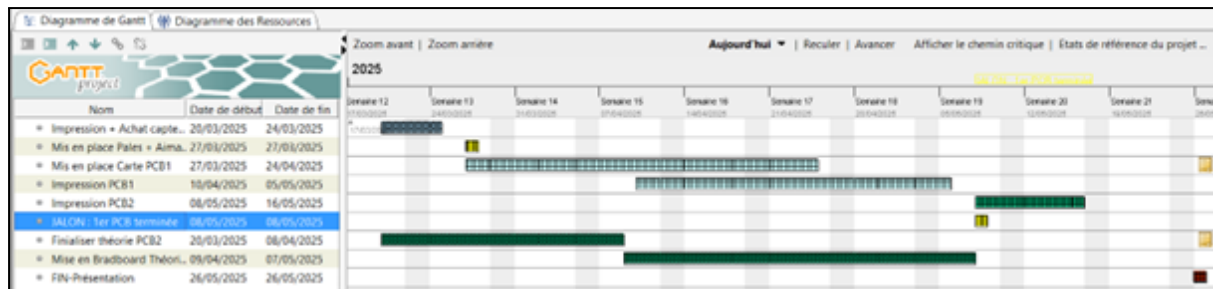
Le reste a été fait en commun.

2) Déroulement chronologique du projet

a) En premier lieu, on a fait des tests pour s'assurer que le capteur est bien fonctionnel. Cela nous a pris énormément de temps(+5 séances). Les capteurs à effets Hall qui nous ont bloqué lors du début de notre projet étaient soit cassés soit trop peu sensibles.

En effet le capteur qui fonctionna était 10 fois plus sensible que ceux d'avant et même avec ce capteur on a dû prendre des aimants en néodyme pour voir clairement une évolution.

b) Diagramme de Gantt (Application utilisée GanttProject) :



Fait milieu mars après avoir enfin réglé nos soucis avec le capteur à effet Hall.

Lizandre devait faire en parallèle un montage électrique qui asservirait sans utiliser la carte STM32 (pas fait).

Évènements marqués sur le diagramme de Gantt :

GANTT project		
Nom	Date de début	Date de fin
● Impression + Achat capteur...	20/03/2025	24/03/2025
● Mis en place Pales + Aiman.	27/03/2025	27/03/2025
● Mis en place Carte PCB1	27/03/2025	24/04/2025
● Impression PCB1	10/04/2025	05/05/2025
● Impression PCB2	08/05/2025	16/05/2025
● JALON : 1ère PCB terminée	08/05/2025	08/05/2025
● Finaliser théorie PCB2	20/03/2025	08/04/2025
● Mise en Bradboard Théorie	09/04/2025	07/05/2025
● FIN-Présentation	26/05/2025	26/05/2025

En parallèle :

- Vérification du bon fonctionnement de chaque composant (potentiomètre, transistor moteur, capteur à effet Hall, ...)

Le Jeudi 22/05/2025 :

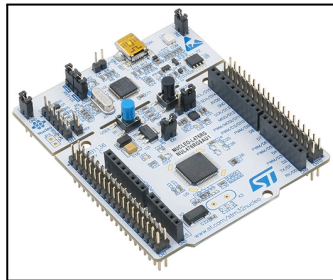
- L'assemblage final
- L'ajout du potentiomètre

- Programmation de l'asservissement

II - Partie technique

II - 1 Choix technologiques

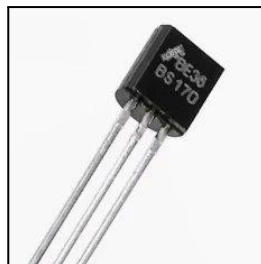
Pour faire notre asservissement on utilisera une carte STM32 NUCLEO-L476RG [La carte utilisée en cours].



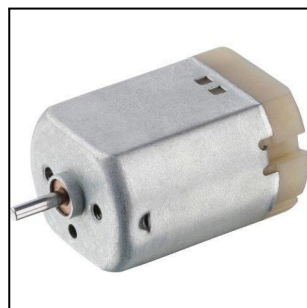
L'asservissement utilisé sera du type tout ou rien.

On considérera que le temps à 5% du moteur est bien plus petit que la vitesse de commutation pour respecter le cahier des charges avec un tel asservissement.

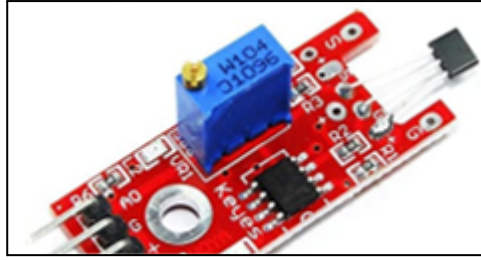
Les commutations vont être gérées par le MOSFET BS170 [Type N enrichi] supportant les courants tirés par le moteur en fonctionnement nominal, soit $\sim 0.1A$.



Le MCC utilisé sera le **Motraxx LFK-280SAV-19170** car il permet d'avoir une vitesse de rotation $> 3000\text{tour.min}^{-1}$ pour une tension de 5V.



Le capteur à effet Hall est monté sur un PCB tout fait car les options de capteurs sans PCB testés ne fonctionnaient pas).



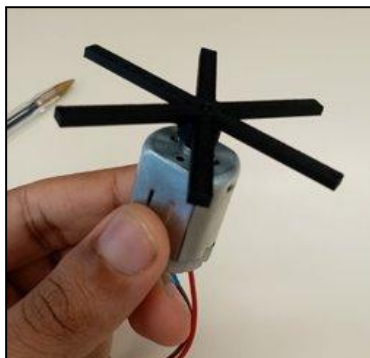
Le capteur à effet Hall a été utilisé car malgré sa faible résolution aux faibles vitesses, on considère que la plage d'utilisation des pâles est au-delà de 15 tours.s⁻¹ (Si le drone tombe, il faudrait que cela soit en 'douceur').

Dans ce cas, cette technologie est plus performante qu'un capteur optique (car sensible à la crasse) ou qu'une dynamo tachymétrique (usure mécanique, durée de vie de 10000 h).

Le capteur à effet Hall n'étant pas très sensible, on a utilisé un aimant en néodyme pour augmenter l'intensité du champ magnétique (un aimant de frigo étant à peine visible).

La modification manuelle de la vitesse à asservir se fait par le biais d'un potentiomètre en série d'une résistance. On veut éviter un court-circuit lorsque le potentiomètre est nul (économie d'énergie).

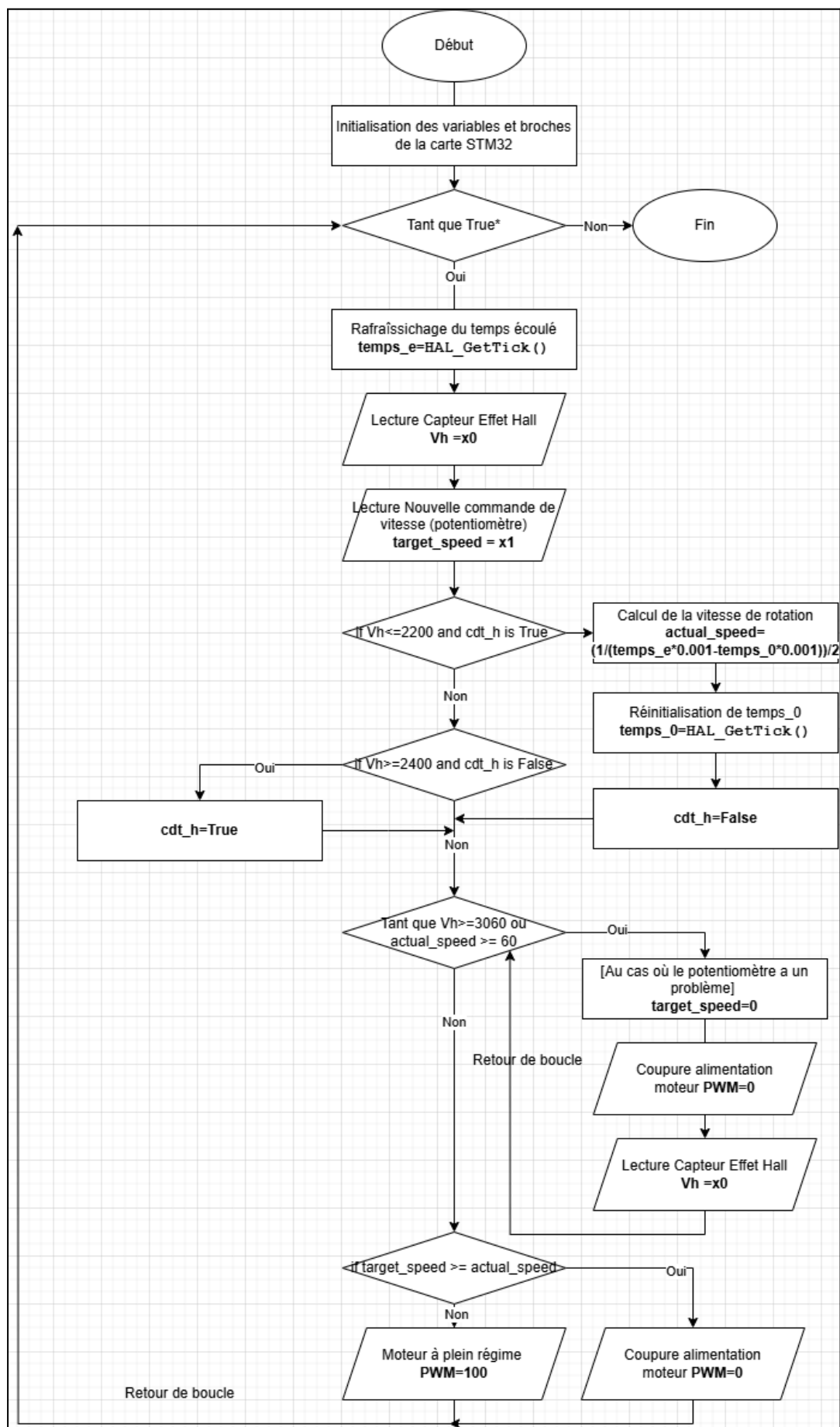
Les pâles ont été imprimées en 3D. On avait besoin d'un support rigide pour notre aimant, des pâles du marché auraient été trop fragiles surtout pour un moteur de cette taille (voir photo).



II - 2 Réalisations

a) La programmation sur la carte STM32

Diagramme de flux



Remarques :

Le diagramme de flux ne suit pas exactement le même cheminement que le code.

Le code est à voir en annexe.

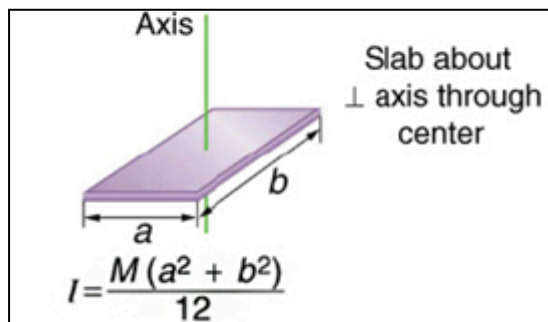
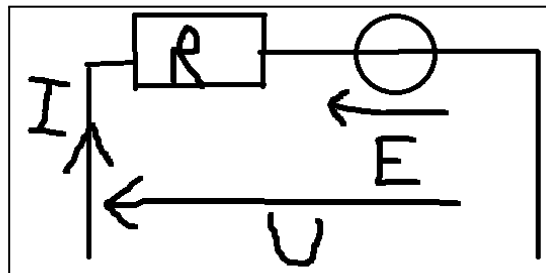
Le code se termine lorsque que la carte n'est plus alimenté d'où * à while True.

La raison pour laquelle on compare V_h à des seuils arbitraires est à voir en d).

b) Preuve théorique et pratique que si l'asservissement tout ou rien fonctionne dans notre cas

Théorie :

On veut montrer en manipulant des ordres de grandeur que l'évolution de la tension sur une mesure est négligeable. D'où une utilisation du tout ou rien justifiée.



AT MAXIMUM EFFICIENCY			
Speed	Current	Torque	Output Power
rpm	A	mNm	W
10.029	0,47	3,81	4,18

Je postule que l'inertie engendrée par les pâles est de l'ordre de 10^{-6} kg.m^2 . En outre, la constante électromotrice est de (Rmoteur négligeable donc $E \sim U$) $K = 8.5 \cdot 10^{-3} \text{ V.s.rad}^{-1}$.

Vu que :

$$J \cdot \frac{d\Omega}{dt} = \Gamma_{pire des cas} = \Gamma_{max} = 0.2K$$

0.2A est le courant maximum que doit passer par le transistor.

On a :

$$\frac{dU}{dt} = \frac{0.2}{J} = 2.10^5 \text{ V.s}^{-1}$$

On en déduit que la variation de vitesse de rotation est de 270 tours.s⁻² dans le pire des cas.

Cependant le rafraîchissement de **actual_speed** (voir diagramme de flux) est dépendante de la vitesse de rotation.

Si on veut 10 tours.s⁻¹, la vitesse à atteindre, étant actualisée à chaque demi-tour, la variation de vitesse entre chaque demi-tour sera de 13.5 tours ce qui est inacceptable. A 20 tours.s⁻¹, on est à 7.25 tours.s⁻¹ tour.

Les résultats théoriques, heureusement, sont pris au pire des cas. Toutefois, l'ordre de grandeur est très conséquent et prouve que cet effet gênera sûrement les "basses vitesses" (0 à 15 tours.s-1).

Pratique :

En mettant le bout de code ci-dessous, on se rend compte qu'un tour de boucle est en moyenne ~25µs (en comptant les cas où on passe par les conditions).

<code>if(i == 10000){</code>	mesure :229
<code> i=0;</code>	mesure :229
<code> printf("mesure :%d\r\n",HAL_GetTick()-mesure);</code>	mesure :229
<code> mesure=HAL_GetTick();</code>	mesure :229
<code>}</code>	mesure :229
<code>else{i=i+1;}</code>	mesure :229

Donc la carte est bien assez rapide pour réagir instantanément à un changement de fréquence. En outre, le MOSFET l'est aussi car son temps d'activation (pour passer de bloquante à passante) est de 10ns.

Si il y a un souci de lenteur, ça ne viendra pas de ces composants.

Ici, on a:

- **target_speed** correspond à la vitesse de rotation souhaitée (c'est le potentiomètre qui le fixe)
- **Pulsewidth (ou PWM dans le diagramme de flux)** % de puissance délivrée au moteur
- **freq OK (ou actual_speed dans le diagramme de flux)** correspond à la vitesse de rotation mesurée par le capteur

freq OK=19	freq OK=35
target_speed :19	target_speed :37
Pulsewidth :100	Pulsewidth :100
freq OK=20	freq OK=35
target_speed :19	target_speed :37
Pulsewidth :100	Pulsewidth :100
freq OK=20	freq OK=33
target_speed :19	target_speed :37
Pulsewidth :0	Pulsewidth :100
freq OK=19	freq OK=38
target_speed :19	target_speed :37
Pulsewidth :0	Pulsewidth :100
freq OK=19	freq OK=33
target_speed :19	target_speed :37
Pulsewidth :100	Pulsewidth :0
freq OK=18	freq OK=38
target_speed :19	target_speed :37
Pulsewidth :100	Pulsewidth :100
freq OK=19	freq OK=33
target_speed :19	target_speed :37
Pulsewidth :100	Pulsewidth :0

A faible vitesse, le capteur à effet Hall n'est pas censé être adapté.

On observe que pour 2 tour.s-1 le moteur va être tellement freiné qu'il va s'arrêter avant de finir son demi tour et donc le capteur ne peut plus détecter un passage des pâles.

A "vitesse moyenne" (ici 19tour.s-1) la précision avec notre méthode est de 2.6%.

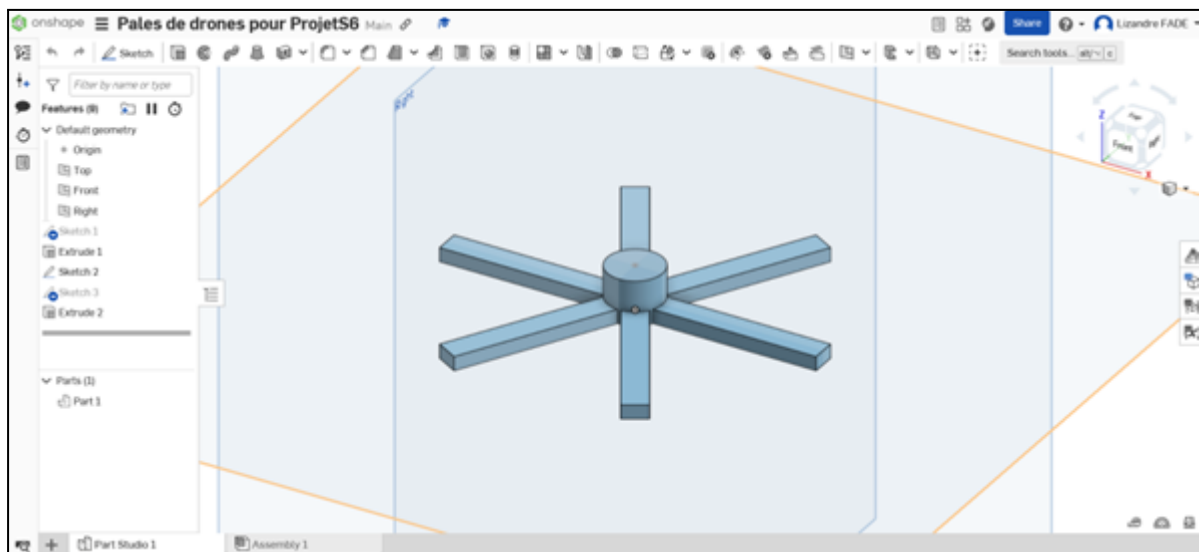
A "haute vitesse" (ici 37 tour.s-1) la précision de notre méthode est de 6.9%. Le cahier n'est pas respecté.

L'asservissement est trop violent, on voit que dès que le **target_speed** est dépassé, le **freqOK** dégringole trop vite. (voir ci-dessous)

```
target_speed :37
Pulsewidth :100
freq OK=38
target_speed :37
Pulsewidth :100
freq OK=33
```

c) Conception des pâles

Les pâles ont été imprimées en 3D après un design conçu sur Onshape (un logiciel de CAO). Cela nous permet de rendre la maquette plus crédible d'un point de vue cinétique (Inertie sur l'arbre moteur augmentée/~ pâles) et de créer un support pour accrocher notre aimant.

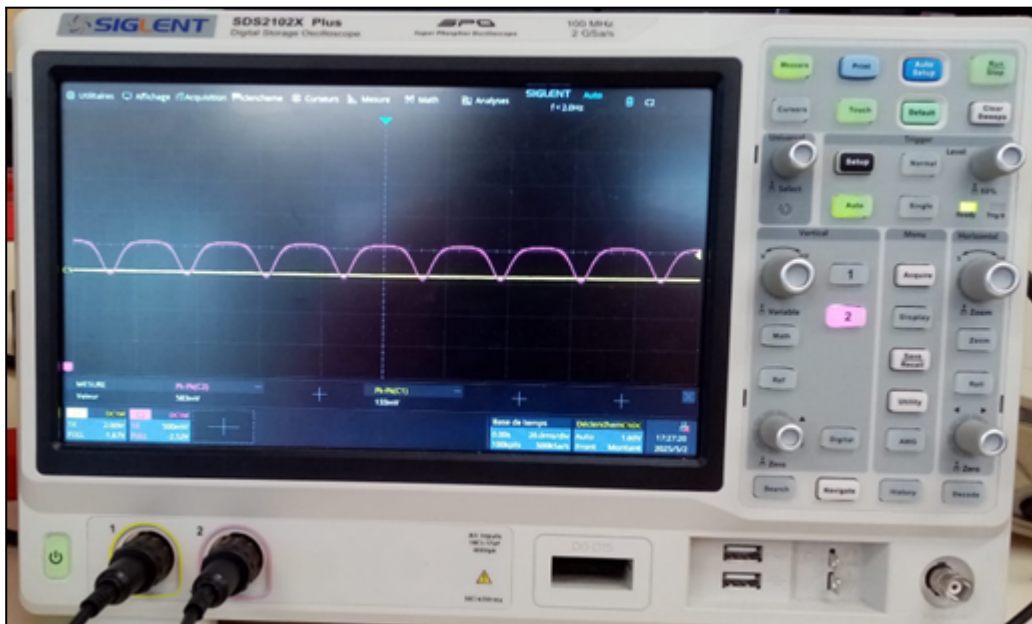


d) Particularité capteur à effet Hall

On veut asservir un MCC selon sa vitesse, il faut décider d'un type d'asservissement (linéaire/tout ou rien) et d'un moyen de récupérer la vitesse et de l'envoyer à la carte STM32.

Un capteur à effet Hall mesure linéairement l'amplitude du champ magnétique (à ne pas confondre avec commutateur magnétique qui se déclenche à un seuil de champ magnétique)

fixé). On va donc placer sur les pôles du moteur (voir I-a) un aimant (en néodyme) qui créera des pics de tension tous les demi-tours de pôles.



La sortie du capteur présente un offset de tension présent qui reste constant si l'alimentation est aussi constante.

Ce signal est ensuite traité par la carte STM32 grâce à l'entrée analogique (PA0). Après avoir choisi arbitrairement des seuils prenant en compte l'offset, on pilote un MOSFET par une sortie analogique PWM en D7.

Il ne reste plus qu'à mettre un PCB pour placer le MOSFET et le Capteur sans breadboard et à coder l'asservissement sur la STM32.

e) Potentiomètre

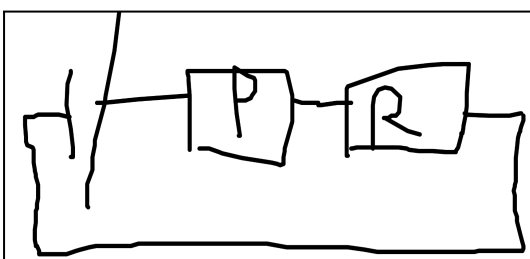
Prendre R en premier permettrait d'atteindre Alim mais pas 0, pour P c'est l'inverse. Vu que on préfère avoir un 0 fixé pour 'éteindre le moteur' on prend P premier.

On a $V_R = P \cdot \text{Alim} / (R + P)$, si $P = 0 \Rightarrow V_R = 0$ / si $P \gg R \Rightarrow V_R \rightarrow \text{Alim}$.

On sait que l'ordre de grandeur de la consommation du courant du moteur est de 0.1 A, on prendra $P = [0, 10\text{k}\Omega]$ et $R = 1000\Omega$ car on veut éviter de gaspiller de l'énergie et ne pas créer un court-circuit si $P = 0\Omega$.

On obtient :

- Au pire des cas $I_{\text{max}} = 5/500 = 0.005 \text{ A}$
- $V_{R\text{max}} \sim 4.55\text{V}$

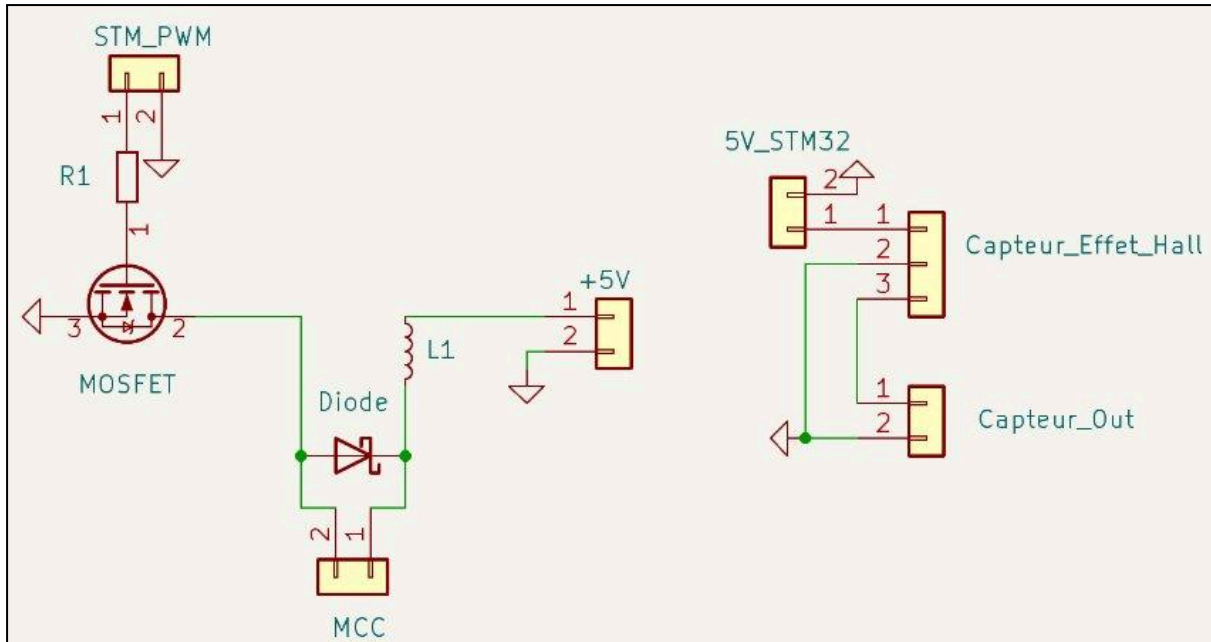


La tension d'alimentation minimale pour pouvoir atteindre la tension maximale mesurable par la carte STM32 est de 3.3V

Résolution de 4096 bits (2^{12}) sur $[0, 3.3\text{V}]$

On veut $V_{Rmax} = Alim$, soit sachant $P=[0,10kOhm]$, on a **Alim = 3.63V** ! Remarque : on a manipulé avec le 3.3V de la carte pour se simplifier la vie.

f) Design PCB



L1 a pour objectif de 'lisser' le signal.

La diode de roue libre a été mise à cause de L1 et des charges inductives présentes dans le MCC. Le courant doit s'écouler en sens inverse lors de la désactivation du MOSFET.

R1 est facultative.

III - Conclusion

a) Vue globale du projet

- Le montage final a bien fonctionné et a bien fait ce que l'on attendait (mais pas du point de vue du cahier des charges).
- Avec un meilleur asservissement, au-dessus de 15 tours.s-1, les capteurs à effet Hall peuvent être considérés comme une alternative cohérente.

b) Les problèmes et leurs éventuelles solutions

i - L'asservissement tout ou rien est trop violent, il faut commander plus précisément le PWM (utiliser les valeurs entre 0 et 100).

Un moyen aurait été de faire un tableau alliant vitesse de rotation et PWM puis de modifier le PWM par incrémentation.

Ainsi si le **target_speed** est modifié, en fonctionnement normal, le moteur sera tout de suite alimenté à la bonne tension.

Le problème est néanmoins le suivant : comment incrémenter le PWM si le milieu est très perturbé (vent intermittent par exemple) ? Cette question je n'ai pas su y répondre cela demanderait de prédire le PWM optimal pour la mesure donnée et mesurée mais je ne sais pas faire.

En bricolant une dichotomie, on aurait pu toutefois mieux respecter le cahier des charges.

ii - On aurait pu mettre un mode sleep au cas où le **target_speed** (voir diagramme de flux) est mis à 0 pour éviter de gâcher de l'énergie.

ANNEXE

Spécifications composants

KY-024 :

<https://www.alldatasheet.fr/datasheet-pdf/pdf/1284502/JOY-IT/KY-024.html>

MOSFET BS170 [Type N enrichi] :

<https://www.farnell.com/datasheets/4459580.pdf>

Motraxx LFK-280SAV-19170 :

<https://www.conrad.fr/fr/p/moteur-universel-a-balais-brushed-motraxx-lfk-280sav-19170-229020.html>

Code final

Code final (fait sur CubeIDE lignes 126-211):

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
//Initialisation
//Code pour commander le MOSFET
TIM_OC_InitTypeDef sConfigOC = {0};
uint8_t PulseWidth = 0; //C'est sur 8 bits mais on utilise que ceux de 0
à100 (décimales)
HAL_TIM_PWM_Start(&htim3,TIM_CHANNEL_2);
sConfigOC.OCMode = TIM_OCMode_PWM1;
sConfigOC.Pulse = PulseWidth; // Set the CCR value (initial duty cycle:
50%)
sConfigOC.OCpolarity = TIM_OCPolarity_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_2);
//Code pour choper les données du capteur (CHANGER VAL ARBITRAIRE)
int val_PA1=0;
int val=0;
int cdt_hysteresis=0;
int last_pic = HAL_GetTick();
int time;
int freq=0;
int val_tested=0; //pour connaitre le nb d'échantillon analysés et voir
int target_speed=0;
int MAX_SPEED=51; //Par expérience vitesse de rotation du moteur à ~5V
//int target_speed=0;
//Mise en route
while (1) {
    //Valeur ADC (capteur à effet Hall)
    val_tested=val_tested+1;
    HAL_ADC_Start(&hadc1);
```

```

    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    val=HAL_ADC_GetValue(&hadc1);
    //Valeur PA1
    HAL_ADC_Start(&hadc2);
    HAL_ADC_PollForConversion(&hadc2, HAL_MAX_DELAY);
    val_PA1=HAL_ADC_GetValue(&hadc2);
    target_speed=(int) (val_PA1*MAX_SPEED/4096); //J'aurais pu mettre un
    coeff en considérant que la tension mesuré ne dépasse pas 3V !
    //Mesure temporelle
    time=HAL_GetTick();
    //Gestion de PWM (ici sConfigOC.Pulse)
    if(target_speed >= freq){
        sConfigOC.Pulse = 100;
        HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC,
TIM_CHANNEL_2);
        //printf("freq VERIF=%d\r\n", freq);
    }
    if(freq > target_speed){
        sConfigOC.Pulse = 0;
        HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC,
TIM_CHANNEL_2);}

    //Raffraîchement d'actual_speed (ici freq)
    if(val <= 2200 && cdt_hysteresis==1){ //Seuil arbitraire déterminé par
l'expérience [IL FAUT FAIRE UN STAND POUR LE MOTEUR !!]
        freq=(int) (1/(time*0.001-last_pic*0.001))/2; //Conversion en
milli
        printf("freq OK=%d\r\n", freq); //Voir la vitesse de rotation
affichée dans la console
        //ça ne fonctionne qu'à haute vitesse (à haute vitesse [+ de 10
tours.s-1]
        //le gain relatif et la perte relative du moteur seront négligeable
[à prouver !])
        //On considère arbitrairement que la plage d'utilisation est au
dessus des 5 tour.s-14
        last_pic=HAL_GetTick();
        cdt_hysteresis=0;
        printf("target_speed :%d\r\n", target_speed);
        printf("Pulsewidth :%d\r\n", sConfigOC.Pulse);
    }
    //PAR SECURITE
    while(val>=3060 || freq >= 60){
sConfigOC.Pulse = 0;
target_speed=0;
freq=0;
HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_2);
        HAL_ADC_Start(&hadc1);
        HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
        val=HAL_ADC_GetValue(&hadc1);
    }

    //Assure l'hystérésis
    if(val >= 2400 && cdt_hysteresis==0){
        cdt_hysteresis=1;}

```

```
/* USER CODE END WHILE */  
/* USER CODE BEGIN 3 */  
}
```